

VisualFlow: An Interpretable Machine Learning Framework with Process Visualization

王扬, 徐行之, 曾华正, 朱博宇

June 17, 2025

Abstract

Interpretable machine learning (ML) is gaining prominence as models become increasingly complex. However, most current tools provide only static model visualizations and lack support for visualizing the dynamic processes behind model training, inference, and explanation. We introduce **VisualFlow**, a customizable, interpretable ML framework that enables *process visualization* across traditional and deep learning models. Implemented using Gradio, VisualFlow offers dynamic insights into model training, internal state changes, and explainable AI (XAI) outputs through custom implementations of LIME and SHAP. This paper presents the design, process visualization capabilities, and interpretability features of VisualFlow, emphasizing its extensibility and contribution to ML process visualizations.

1 Introduction

With the growing complexity and deployment of machine learning models in high-stakes domains, there is a rising demand for visualization and interpretability. While many ML frameworks offer metrics to evaluate performance, they often lack the ability to dynamically visualize internal behaviors and explain model predictions.

Existing visualization tools suffer from three main limitations:

- They provide only **static visualizations**, insufficient to understand the evolution of model training or inference steps.
- They lack support for **custom or locally-deployed models**, which are common in practical development workflows.
- Built-in visualization components focus mostly on **high-level training metrics**, without providing insights into internal states or reasoning processes.

To address these challenges, we present **VisualFlow**, an interpretable ML framework designed to visualize training and inference processes and explain model predictions.

Our contributions are threefold:

1. We design a **process-oriented visualization system** that captures dynamic changes in training loss, weight evolution, feature maps, and intermediate states.

2. We implement **custom explainability methods**—LIME and SHAP—from scratch, allowing deep insight into model decisions.
3. We support a wide range of **traditional and deep learning models**, including Linear Regression, SVM, CNN, U-net, and MobileNet_v2, enabling extensibility across modeling paradigms.

VisualFlow is built on Gradio, enabling seamless web-based interaction with models and visual explanations. This integration of implementation, visualization, and interpretability into a single framework makes VisualFlow a valuable tool for researchers and practitioners seeking process visualization in machine learning systems.

2 Framework Overview

VisualFlow is implemented using Gradio and is organized into three core modules: model modules, visualization modules, and explanation modules. These components work together to support end-to-end model interpretation and process visualization.

2.1 Model Modules

Based on a modular ML framework, VisualFlow incorporates a range of widely used machine learning and deep learning models. The supported models include:

- **Traditional Models:** Support Vector Machines (SVM), Linear Regression, K-means Clustering
- **Neural Networks:** Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), U-net, Diffusion Models, MobileNet_v2

These models provide a comprehensive foundation for both basic algorithm education and advanced interpretability research.

2.2 Visualization Modules

VisualFlow provides real-time visualization of model behavior during both the training and inference stages. Key functionalities include:

- **Training Phase:**
 - Visualization of loss curve progression over epochs
 - Dynamic tracking of weight distribution changes
 - Logging and display of training process information
- **Inference Phase:**
 - Feature map visualization to explore intermediate representations
 - Output spectrum visualization for classification or regression results

These features offer insights into model learning dynamics and decision patterns.

2.3 Explanation Modules

To enhance interpretability, VisualFlow integrates two popular explainable AI (XAI) methods implemented from scratch:

- **SHAP**: Applied to structured datasets such as the Iris dataset in combination with models like linear regression, to assess the contribution of each feature to the prediction.
- **LIME**: Used for image classification tasks (e.g., ImageNet with MobileNet_v2), identifying and visualizing the top contributing regions in an image.

These methods provide localized, human-readable interpretations of model predictions, supporting both educational use and trustworthiness assessment.

3 Visualization Examples

To demonstrate the capabilities of VisualFlow, this section presents several representative visualization examples covering both traditional and deep learning models. These examples highlight how our framework captures the internal processes of different models during training and inference.

3.1 K-Means

This section visualizes the iterative optimization process of the K-Means clustering algorithm in VisualFlow. The optimization path illustrates the step-by-step solution of the clustering problem.

In each frame, the visualization illustrates two main operations:

1. **Assignment Step**: Each data point is assigned to the nearest centroid, and the points are color-coded based on their assigned cluster.
2. **Update Step**: The centroids are recalculated as the mean of all points within their respective clusters. The new positions of the centroids, marked with an 'x', are plotted in the subsequent frame.

As shown in Figure 2, the algorithm converges by iteratively adjusting cluster boundaries and centers.

3.2 SVM

This section presents a visualization of the margin maximization process in Support Vector Machines (SVM).

The optimization path illustrates the step-by-step solution of the margin maximization problem.

For each step, the process is as follows:

- A linear SVM model is trained using only the current subset of data.

- A frame is generated that shows a scatter plot of the *entire* dataset, ensuring a consistent background for comparison.
- The **decision boundary** derived from the model trained on the subset is overlaid on the plot.

The resulting animation displays how the separating hyperplane shifts and adjusts, providing insight into how the SVM’s solution stabilizes. Visualization example is showed in Figure 1.

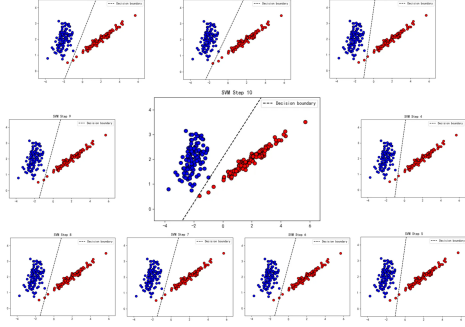


Figure 1: SVM visualization example

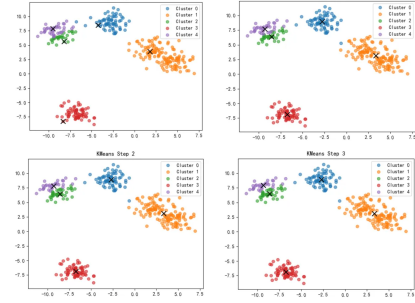


Figure 2: K-Means visualization example

3.3 CNN

This section details the design and implementation of the CNN module, which uses a fixed architecture trained on the MNIST dataset. The module emphasizes comprehensive training and inference visualizations to enhance understanding and interpretability of convolutional neural networks.

3.3.1 Task Setting

The CNN module is pre-configured with a standard architecture suitable for MNIST digit classification, including convolutional, pooling, and fully connected layers. Training runs are performed on the MNIST dataset, and all training parameters such as learning rate and epochs are fixed to ensure reproducibility. This setup simplifies experimentation and focuses the visualization on model behavior and internal dynamics.

3.3.2 Training Visualization

- **Training Log and Loss Curve:** VisualFlow records and displays detailed training logs including epoch-wise loss and accuracy metrics. A loss curve graph plots the model’s convergence trajectory, allowing users to track training progress and diagnose issues like underfitting or overfitting.
- **Weight Visualization:** The evolution of convolutional filter weights is visualized as images or heatmaps at different epochs. Users can observe how filters learn to detect edges, textures, and digit-specific features as training progresses.

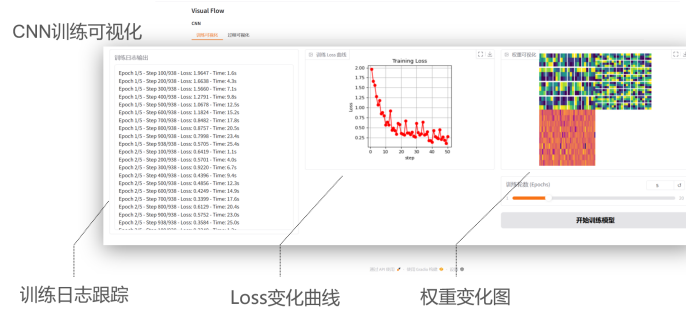


Figure 3: CNN Training-phase visualization

3.3.3 Inference Process Visualization

- **Layer-wise Feature Map Extraction:** For each input digit image, VisualFlow extracts and displays the feature maps from all convolutional layers. These feature maps illustrate how the network transforms raw pixel data into progressively more abstract representations.
- **Final Prediction Visualization:** The output probabilities over the 10 digit classes are shown as a bar chart alongside the predicted label, providing clear insight into the model's confidence and decision.

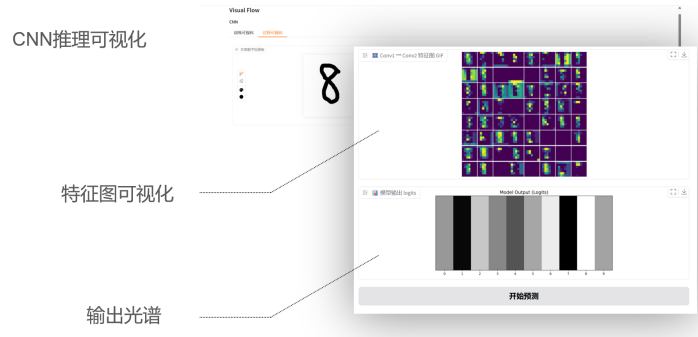


Figure 4: CNN Inference-phase visualization

Through these visualizations, VisualFlow offers an intuitive and interactive understanding of CNN training dynamics and inference behavior, making deep learning models more transparent and accessible.

3.4 MLP

This section describes the design and implementation of the MLP module, which serves as a core component for network visualization and interpretability.

3.4.1 Task Setting

Users can select from preloaded datasets such as MNIST and Circle, or upload their own datasets. Once a dataset is selected, the system automatically configures input and output

dimensions. Users can customize the hidden layer dimensions. This process lowers the barrier for experimentation and enables rapid prototyping of MLP architectures.

3.4.2 Visualization Modules

The visualization modules are divided into two major categories: training visualization and inference-time process visualization.

Training Visualization

- **Loss Curve Plotting:** The system plots the loss value across training epochs, allowing users to monitor convergence and detect underfitting or overfitting trends.
- **Neural Network Structure Visualization:** The architecture is represented as a graph: neurons are visualized as circles and weights as connecting lines. Two dynamic visual effects are supported:
 - *Dynamic Weight Visualization:* The evolution of connection weights is displayed as animations in which line thickness indicates weight magnitude. This helps users observe how the network gradually adjusts its internal representations and identifies crucial connections during learning.
 - *Dynamic Activation Visualization:* Neuron activation values change in response to different inputs and are visualized through the color intensity of each neuron node. This enables users to see which parts of the network are more active under various input conditions, providing intuition about feature extraction and representation learning.

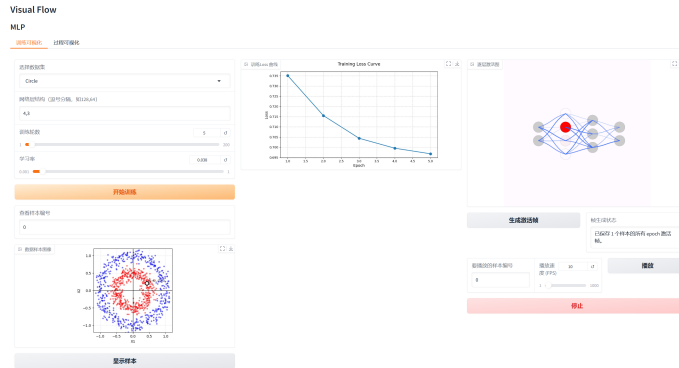


Figure 5: Training-phase dynamic activation visualization

Inference Process Visualization

- **Custom Input Testing and Output Explanation:** Users can enter custom test samples to observe:
 - The probability distribution over all classification candidates (as a bar chart)
 - The final predicted label

- A step-by-step visualization of how the network reaches the decision, including intermediate activations and contribution of each layer
- **Inference-Time Activation Animation:** Similar to training, the activations of neurons are shown dynamically as input changes. This helps reveal how the network processes specific inputs and highlights the internal decision-making path.

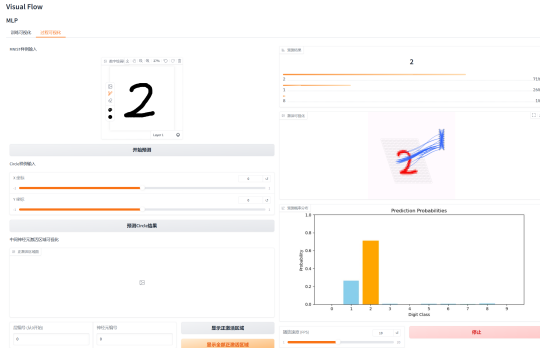


Figure 6: Inference-phase dynamic activation visualization

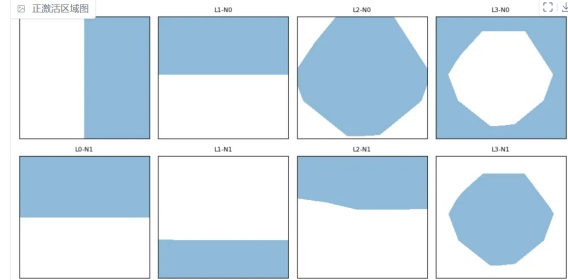


Figure 7: Neuron feature maps highlighting decision regions

- **Neuron Feature Maps and Class Contribution Regions:** VisualFlow supports generation of per-neuron feature influence maps. These maps reveal which neurons are responsible for partitioning input space into class-related regions, improving interpretability of learned representations.

4 Interpretability Examples

This section introduces representative explanation techniques—SHAP and LIME—that help visualize how different input features or image regions influence model predictions. These methods support both structured data and image data, offering insight into model behavior across modalities.

SHAP measures feature importance by enumerating all possible combinations of features and compute the average of performance drop when the feature is removed.

LIME (Local Interpretable Model-agnostic Explanations) constructs interpretable local models to approximate how a complex model behaves around specific data points.

4.1 SHAP

We employ SHAP to interpret a multiclass Linear Regression model trained on the Iris dataset. The model correctly predicts the sample as virginica, with feature contributions shown across four attributes. Model hyperparameters used were $C = 1.0$, $\text{penalty} = 2$

The figure on the left illustrates the estimated Shapley values for Sample 0, while the right-side plot visualizes the model prediction trend.

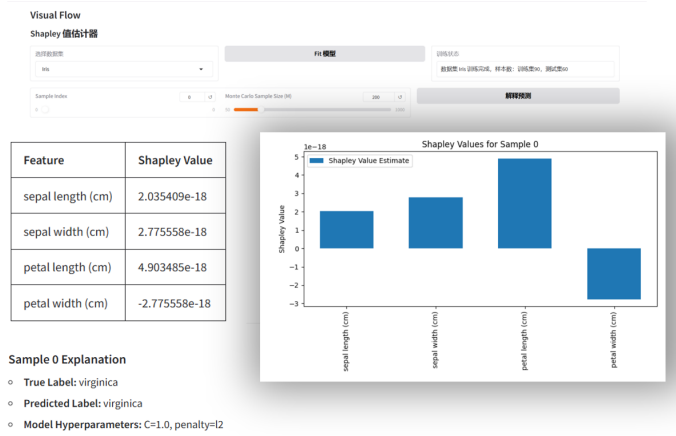


Figure 8: SHAP explanation for Sample 0 (Iris dataset)

4.2 LIME

Using LIME on MobileNetv2, VisualFlow visualizes the top 50 most influential regions contributing to the model’s classification. We apply LIME to interpret predictions made by a MobileNetv2 model on natural image data. As shown in the figure below, LIME highlights the **top 50 super-pixel regions** that contribute most significantly to the model’s classification decision.

The interpretation reveals that key structural features such as the *bus head*, *windows*, and *wheels* play a decisive role in the classification outcome. This localized visual explanation offers transparency for individual predictions in deep image models.

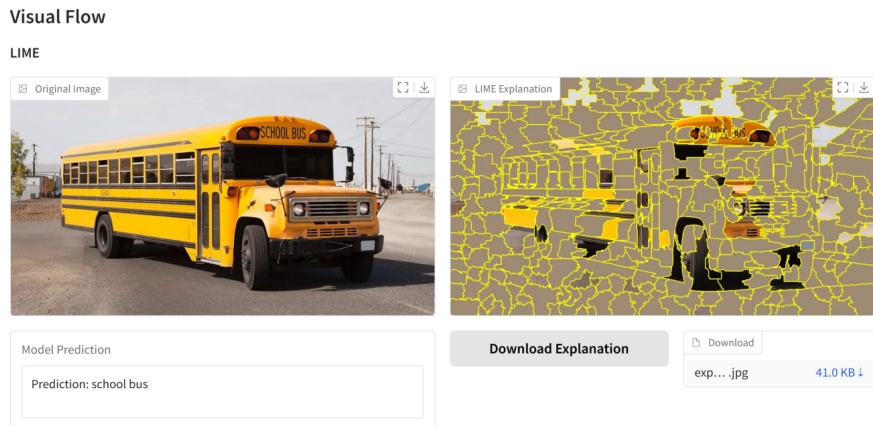


Figure 9: LIME explanation for ImageNet

5 Future Work

While VisualFlow demonstrates promising capabilities in process visualization and interpretability, several limitations remain to be addressed in future iterations:

- **Limited extensibility for complex models:** For models like CNNs, our current implementation relies on built-in feature map extraction. This design restricts extensibility and model-agnostic support. In future work, we plan to introduce hook-based mechanisms that allow flexible integration with arbitrary model architectures and support user-defined feature map visualizations.
- **Lack of structural customization:** At present, users can only modify specific hyperparameters of predefined models, without the ability to freely define or compose model architectures. We aim to incorporate structural customization capabilities, enabling users to build and visualize their own networks interactively.
- **Interactive process control:** Inspired by software debugging paradigms, we envision a mechanism akin to setting breakpoints in training or inference workflows. This feature would allow users to pause at specific layers or steps, inspect intermediate representations, and resume execution, thereby enhancing the interpretability and controllability of the ML pipeline.

6 Conclusion

VisualFlow presents a unified framework that seamlessly integrates model implementation, dynamic training and inference visualization, and interpretable prediction explanation. By addressing critical limitations in existing machine learning platforms, VisualFlow enhances both transparency and usability for a broad range of users, including researchers, educators, and developers.

Our framework contributes to the field in the following key ways:

- Offering **interpretable dynamic visualizations**
- Providing **implementations** of popular XAI methods
- Enabling **extensibility** for various ML and XAI techniques

By combining interpretability, process transparency, and model modularity, **VisualFlow** lays the foundation for a new generation of machine learning tools that are not only powerful but also trustworthy. Future work will focus on expanding support for custom model architectures, enhancing interaction mechanisms, and integrating richer visualization features. Ultimately, we hope that VisualFlow can serve as a practical, extensible, and educational platform for advancing the understanding of machine learning models.

References

- [1] Gradio. *Gradio: Build Machine Learning Web Apps in Python*. Available at: <https://www.gradio.app/>. Accessed June 16, 2025.
- [2] Marco T. Ribeiro, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You?: Explaining the Predictions of Any Classifier." In *Proceedings of the 22nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.
- [3] Scott M. Lundberg and Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. In *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774, 2017.
- [4] Corinna Cortes and Vladimir Vapnik. *Support-Vector Networks*. *Machine Learning*, 20(3):273–297, 1995.
- [5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, LNCS 9351, pp. 234–241. Springer, 2015.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. In *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.
- [7] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.